User Website Features Report

Project: Church / Ministry Organization Website

This report explains every user-side feature under two headings:

1. How It Works: What visitors can do and how they use the feature.
2. Technically How It Works: How developers implement the feature using the frontend, backend, database, and external services.

1. Home Page

How It Works

- Displays the organization's name, logo, introduction, and welcome message.
- Shows featured sermons, upcoming events, service timings, and announcements.
- Provides quick links to important pages.

Technically How It Works

- The frontend uses reusable components to display homepage sections.
- The backend API retrieves published content from the database.
- Images are optimized and cached to improve loading speed.
- Administrators control which content appears as featured content.

2. About Us

How It Works

- Explains the organization's history, mission, vision, and beliefs.
- Helps visitors understand the purpose of the organization.

Technically How It Works

- Content is stored in a database or content management system.
- The frontend retrieves and displays the published information.
- Administrators can update the content without modifying application code.

3. Leadership

How It Works

- Displays leaders' names, photographs, designations, and biographies.
- Allows visitors to learn about the organization's leadership team.

Technically How It Works

- Leader profiles are stored as structured records in the database.
- Images are stored in media storage, with their references saved in the database.

- The frontend displays each profile using a reusable profile component.

4. Service Timings

How It Works

- Displays worship service names, languages, days, times, and locations.
- Helps visitors plan their attendance.
- Shows schedule changes when administrators publish updates.

Technically How It Works

- Service details are stored in a services database table.
- The backend API returns active service schedules.
- The frontend formats dates and times for display.
- Published announcements can communicate temporary changes.

5. Ministries

How It Works

- Displays available ministries, groups, and their activities.
- Provides descriptions and relevant contact information.
- Helps visitors discover groups they may wish to join.

Technically How It Works

- Each ministry is stored as a database record.
- The backend provides ministry listing and detail endpoints.
- The frontend displays ministry cards and individual detail pages.
- Images and contact information are retrieved alongside the relevant records.

6. Sermons & Messages

How It Works

- Allows visitors to browse recorded sermons and messages.
- Displays video titles, speakers, dates, and descriptions.
- Lets visitors select and watch a sermon.

Technically How It Works

- Sermon metadata is stored in a sermons database table.
- Video files can be hosted on YouTube or another video platform.
- The application stores the video URL or provider video ID.
- The frontend embeds the video player and displays the sermon information.

- Pagination and category filters help visitors browse large collections.

7. Live Streaming

How It Works

- Provides access to live worship services or other online gatherings.
- Visitors select a live-stream link to watch the service.

Technically How It Works

- The administrator configures a live-stream URL.
- The backend provides the published URL to the frontend.
- The frontend displays a watch button or embedded video player.
- If supported, the application can show whether a stream is currently live.

8. Events

How It Works

- Displays upcoming events, dates, times, locations, and descriptions.
- Allows visitors to open individual event pages.
- Keeps completed events separate from upcoming events.

Technically How It Works

- Event information is stored in an events database table.
- The backend retrieves published events and filters them by date.
- The frontend displays event cards and detail pages.
- Event images are retrieved from media storage.

9. Event Registration

How It Works

- Allows visitors to register for events when registration is required.
- Collects necessary information, such as name and contact details.
- Displays a confirmation after successful registration.

Technically How It Works

- A registration form submits data to the backend API.
- The backend validates the data and checks registration limits.
- Valid registrations are saved in a registrations table.
- Duplicate submissions and capacity limits are handled by server-side rules.
- A confirmation can be displayed or sent by email.

10. Gallery

How It Works

- Displays photographs and videos from services, meetings, and other activities.
- Allows visitors to browse images and open them for a larger view.

Technically How It Works

- Images and videos are stored in media storage or an external media platform.
- Their URLs, captions, and categories are stored in the database.
- The frontend displays a responsive gallery grid.
- Lazy loading loads images as visitors scroll, reducing initial page load time.

11. Blog

How It Works

- Displays articles, updates, and educational content.
- Allows visitors to open and read complete articles.
- Organizes articles by topic or category.

Technically How It Works

- Articles are stored in a posts or articles database table.
- The backend returns published articles and their metadata.
- Each article has a unique URL or slug.
- The frontend displays the article, publication date, author, and images.
- SEO metadata helps search engines index individual articles.

12. Prayer Request

How It Works

- Allows visitors to submit prayer requests through a form.
- Displays a confirmation after successful submission.
- Can offer an optional confidential submission method.

Technically How It Works

- The frontend submits the request securely to the backend.
- The backend validates input and applies spam protection.
- Requests are stored in a prayer_requests table.
- Access is restricted to authorized personnel.

- Sensitive information is protected through access controls and appropriate data-handling practices.

13. Online Donations

How It Works

- Explains how visitors can contribute financially.
- Provides supported payment options, such as UPI or an online payment gateway.
- Displays a payment result and, where supported, a donation receipt.

Technically How It Works

- The frontend initiates a payment request through the backend.
- The backend creates a transaction with the payment provider.
- The provider processes the payment.
- A verified server-side response or webhook updates the transaction status.
- The application records the transaction reference and verified status.
- Receipts are generated only after payment confirmation.

14. Contact Us

How It Works

- Displays contact numbers, email addresses, and a contact form.
- Allows visitors to send questions or general messages.
- Provides direct call and email links.

Technically How It Works

- Contact information is retrieved from website settings.
- The contact form submits data to the backend API.
- The backend validates and stores messages in a contact_messages table.
- Email notifications can alert authorized staff about new submissions.
- Rate limiting and spam protection reduce abuse.

15. Location & Directions

How It Works

- Displays the organization's address and location on a map.
- Allows visitors to open directions to the physical location.

Technically How It Works

- The address and map coordinates are stored in website settings.

- A map service or embedded map displays the location.
- A directions link opens a supported maps application.
- An external map service may require an API key, depending on the implementation.

16. Social Media Links

How It Works

- Provides links to the organization's official social media accounts.
- Allows visitors to follow the organization or view its social content.

Technically How It Works

- Social media URLs are stored in website settings.
- The frontend displays reusable social media icons and links.
- Links open the relevant platform.
- Social media feeds can be embedded separately if the provider supports it.

17. Mobile App Links

How It Works

- Provides Android and iOS application download links when mobile apps are available.
- Allows visitors to access the organization's mobile application.

Technically How It Works

- Store the official app store URLs in website settings.
- The frontend displays appropriate download links.
- Device-aware links can direct visitors to the correct store.
- Deep linking can open specific application pages when configured.

18. Search

How It Works

- Allows visitors to find sermons, articles, events, ministries, and other website content.
- Displays relevant results based on the search terms.

Technically How It Works

- The frontend sends the search query to the backend.
- The backend searches indexed database fields.
- Search results are filtered to include only publicly available content.
- Pagination and indexes improve performance.
- Bible verse search can use a separate search function tailored to Bible text.

19. Responsive Design

How It Works

- Adapts the website to mobile phones, tablets, laptops, and desktop screens.
- Keeps menus, text, images, and forms usable on different screen sizes.

Technically How It Works

- CSS media queries and responsive layout systems adjust the page layout.
- Flexible grids, responsive images, and scalable typography support different screens.
- Touch-friendly controls improve mobile usability.
- Browser testing checks layout and functionality across screen sizes.

20. Notifications & Announcements

How It Works

- Displays important notices, service timing changes, and organizational updates.
- Shows announcements on the homepage or a dedicated announcements page.
- Can highlight urgent notices and hide expired announcements automatically.

Technically How It Works

- Announcements are stored in an announcements database table.
- Each record contains a title, message, publication date, expiry date, and status.
- The backend returns announcements that are published and currently active.
- The frontend displays them in a banner or announcement section.
- A scheduled task or server-side date check handles expiry.
- Email, browser push, or mobile push notifications can be added separately.

21. Online Bible

How It Works

- Allows visitors to read the Bible in English (King James Version) and Telugu.
- Lets users select a Bible version, testament, book, and chapter.
- Displays the verses in a readable format on mobile and desktop.

Technically How It Works

- Bible text is obtained from a properly licensed source or API.
- Bible content can be stored in a database or retrieved from an external API, depending on licensing and technical requirements.
- The frontend provides version, book, and chapter selectors.

- The backend retrieves the requested passage and returns the verses.
- The application distinguishes Bible versions so English KJV text and Telugu text are not mixed.
- Offline reading can be implemented by storing permitted Bible content locally in the browser or application.

22. Bible Search & Navigation

How It Works

- Allows users to browse Old Testament and New Testament books.
- Lets users navigate directly to a specific chapter or verse.
- Allows users to search for words or phrases within the Bible.

Technically How It Works

- Bible books, chapters, and verses use structured identifiers.
- The frontend provides book and chapter selectors and a search field.
- The backend queries the selected version for the requested passage.
- Search can use database indexes or a full-text search engine.
- Search results include the book, chapter, verse number, and matching text.
- Search is restricted to the Bible versions available in the application.

23. Bible Bookmarks

How It Works

- Allows users to save favorite Bible verses.
- Provides a dedicated list of bookmarked verses.
- Lets users reopen or remove saved bookmarks.

Technically How It Works

- For a website without accounts, bookmarks can be stored in browser local storage or IndexedDB.
- Each bookmark contains the Bible version and verse reference.
- The frontend reads saved bookmarks when the user returns to the application.
- If cross-device synchronization is required, user accounts and a backend database will be needed.
- Bookmarks must identify the Bible version to ensure the correct verse is displayed.

24. Daily Bible Verse

How It Works

- Displays a selected Bible verse each day.

- Shows the verse reference and Bible version.
- Allows visitors to navigate to the full chapter for additional context.

Technically How It Works

- Daily verse references are stored in a database or configured through an approved content source.
- The backend selects the verse assigned to the current date.
- The frontend displays the verse and its reference.
- A consistent date and timezone policy prevents the displayed verse from changing unexpectedly.
- Administrators can manage daily verses if this feature is included in the admin dashboard.

25. General Technical Features

How It Works

- Keeps the website secure, responsive, reliable, and fast.
- Supports data management, troubleshooting, and future improvements.

Technically How It Works

Feature	**Technical implementation**
Database Management	Stores structured content and application records.
API Integration	Connects the frontend, backend, database, and external services.
Security	Uses HTTPS, input validation, secure authentication, and access controls.
Performance Optimization	Uses caching, image optimization, and efficient database queries.
SEO Optimization	Provides page metadata, canonical URLs, and XML sitemaps.
Backup & Recovery	Creates backups and supports restoration procedures.
Error Monitoring	Records errors and supports debugging.
Accessibility	Supports keyboard navigation, readable contrast, and screen readers.

Important implementation note: Because the Bible feature needs both KJV and Telugu content, confirm the source and its licensing before selecting the storage or API approach. Also, since the website is planned without user accounts, bookmarks will remain on the user's device unless account-based synchronization is added later.